

Agosto 14, 2026

Patrones de Software

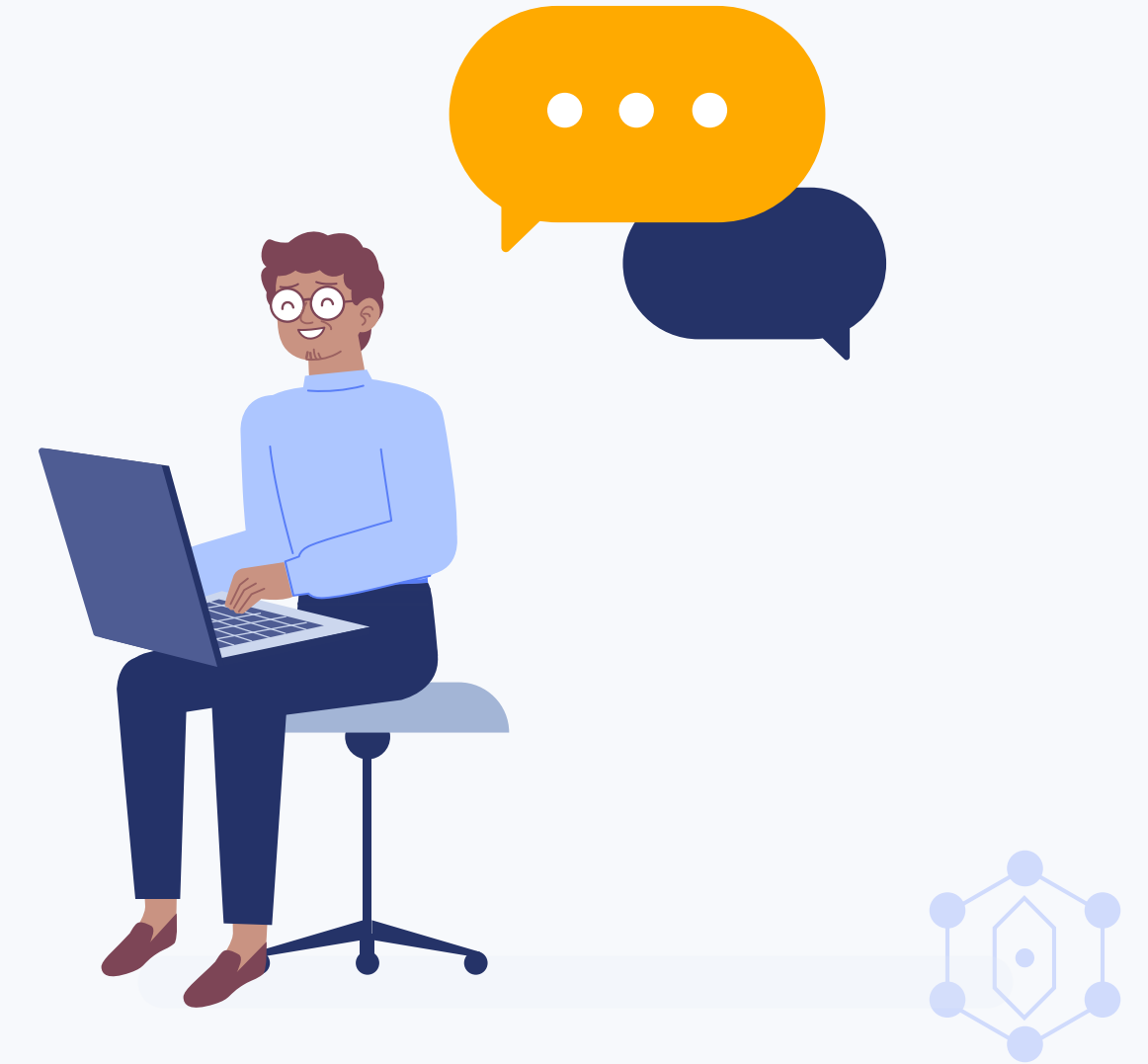
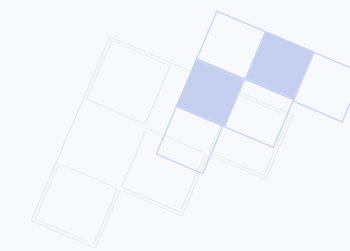
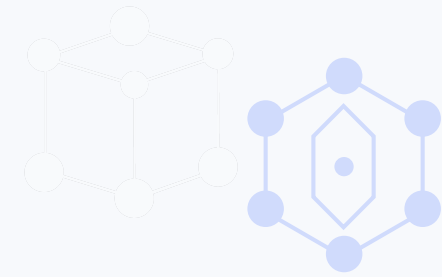
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto



Recordemos

Conceptos vistos en la anterior clase



**Intro a la
Asignatura**

**¿Que son Patrones
de SW?**

**Ventajas de Utilizar
Patrones de SW**

Tipos de Patrones de Software



Tipos de Patrones

Existen tres grandes grupos en el diseño de software



Patrones Creacionales





Definiciones



Son un conjunto de patrones de diseño que se centran en cómo **crear** **instancias de objetos de manera eficiente y flexible**.

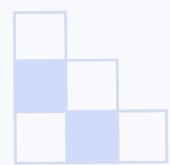


Estos patrones abordan la **creación de objetos de diferentes formas** y contextos, proporcionando soluciones para problemas comunes relacionados con la instanciación de clases y la gestión de objetos.



Patrones Creacionales

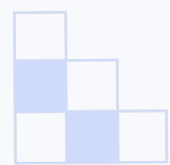
- ✓ **Patrón Singleton:** Este patrón asegura que una clase tenga solo una instancia y proporciona un punto global de acceso a esa instancia. Se utiliza cuando se necesita una única instancia para controlar algún recurso compartido, como una configuración de aplicación o una conexión a una base de datos.
- ✓ **Patrón Factory Method:** Define una interfaz para crear objetos en una superclase, pero permite a las subclases cambiar el tipo de objeto que se crea. Proporciona una forma de delegar la creación de objetos a las subclases, lo que facilita la extensión y el mantenimiento del código.





Patrones Creacionales

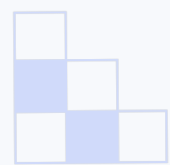
- ✓ **Patrón Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Permite crear objetos que están diseñados para funcionar juntos sin acoplar el código a clases concretas.
- ✓ **Patrón Builder:** Separar la construcción de un objeto complejo de su representación, permitiendo la creación de diferentes tipos y representaciones del mismo objeto. Se utiliza cuando un objeto puede tener varias formas posibles de construcción.





Patrones Creacionales

- ✓ **Patrón Prototype:** Permite la creación de nuevos objetos duplicando un objeto existente, llamado prototipo. Se utiliza cuando la creación de objetos a partir de cero es costosa o compleja.
- ✓ **Patrón Object Pool:** Mantiene un grupo de objetos reutilizables para reducir el costo de creación y destrucción de objetos. Útil cuando crear objetos es costoso y se necesita un gran número de ellos a lo largo del tiempo.
- ✓ **Patrón Lazy Initialization:** Retrasa la creación de un objeto hasta el momento en que realmente se necesita. Puede mejorar el rendimiento y la eficiencia al evitar la creación prematura de objetos.



Patrones Estructurales





Definiciones



Se centran en cómo **componer clases** y **objetos** para formar estructuras más grandes y complejas.

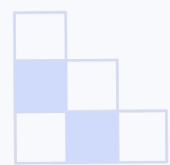


Estos patrones abordan la **organización** y **composición** de **objetos** para lograr sistemas flexibles y extensibles, al tiempo que reducen el acoplamiento entre las partes del sistema.



Patrones Estructurales

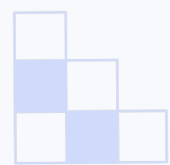
- ✓ **Patrón Adapter:** También conocido como "Wrapper", este patrón permite que interfaces incompatibles trabajen juntas. Actúa como un intermediario entre dos interfaces que no pueden comunicarse directamente.
- ✓ **Patrón Bridge:** Separa una abstracción de su implementación, permitiendo que ambas evolucionen independientemente. Útil cuando hay múltiples dimensiones de variabilidad en una jerarquía de clases.
- ✓ **Patrón Composite:** Permite componer objetos en estructuras de árbol para representar jerarquías. Los objetos individuales y compuestos se tratan de manera uniforme, lo que facilita la manipulación de la estructura.





Patrones Estructurales

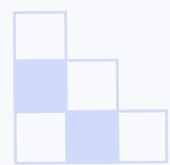
- ✓ **Patrón Decorator:** Agrega responsabilidades adicionales a objetos de manera dinámica y flexible. Es una alternativa a la herencia para extender la funcionalidad de clases.
- ✓ **Patrón Facade:** Proporciona una interfaz simplificada para un conjunto de interfaces más complejas en un sub-sistema. Ayuda a reducir la complejidad y acoplamiento al exponer una interfaz unificada.
- ✓ **Patrón Flyweight:** Minimiza el uso de memoria al compartir de manera eficiente objetos que se utilizan en múltiples contextos. Útil cuando hay una gran cantidad de objetos similares y se busca ahorrar recursos.



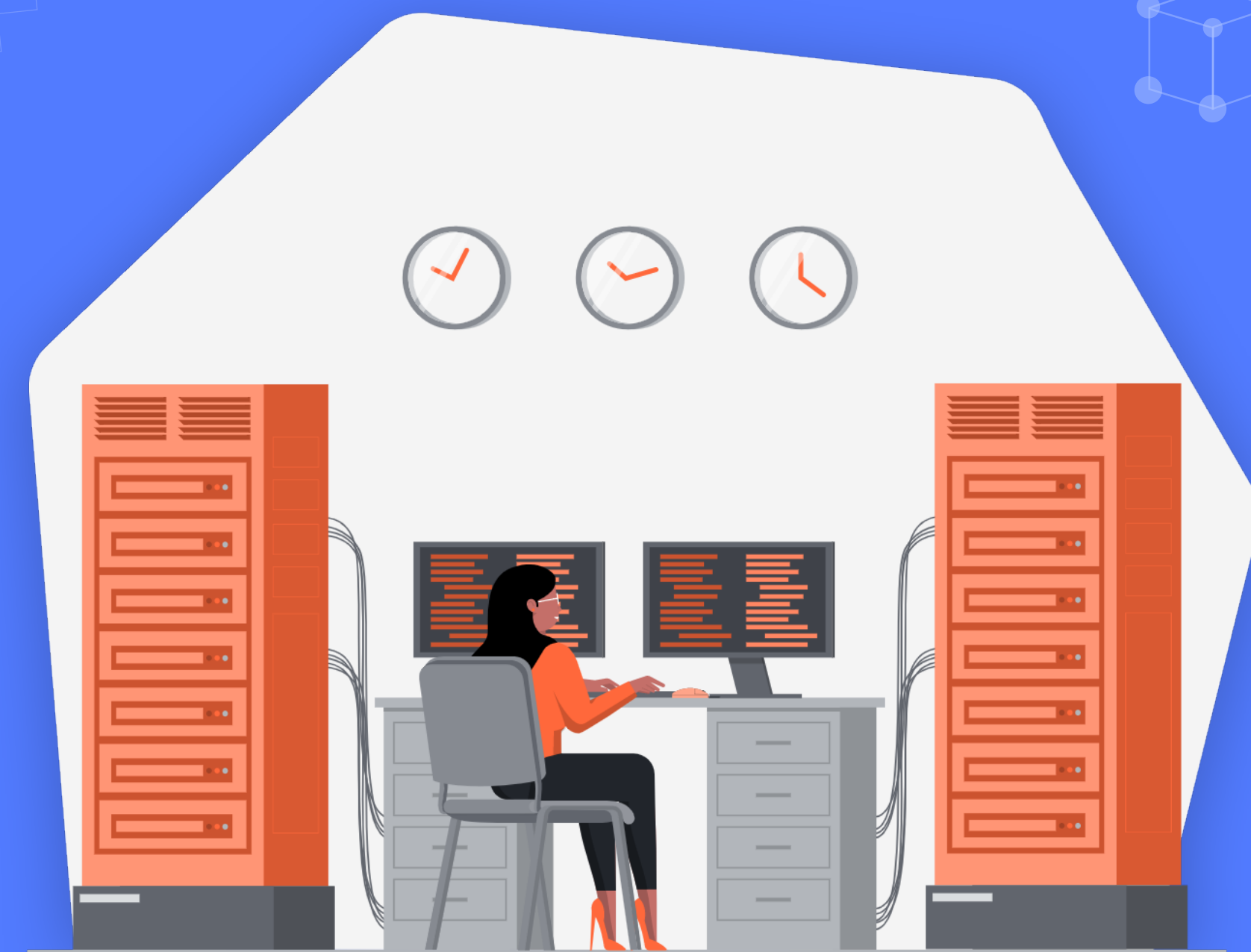


Patrones Estructurales

- ✓ **Patrón Proxy:** Actúa como un representante o sustituto de otro objeto para controlar el acceso a él. Puede utilizarse para agregar funcionalidad adicional o controlar el acceso a objetos sensibles.
- ✓ **Patrón Memento:** Permite capturar y restaurar el estado interno de un objeto sin violar su encapsulación. Útil para implementar funciones "deshacer" o "restaurar" en una aplicación.



Patrones De Comportamiento





Definiciones



Se centran en cómo las clases y objetos **interactúan** y **colaboran** entre sí para lograr comportamientos y funcionalidades específicos en un sistema de software.

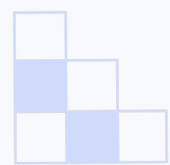


Ayudan a definir las **formas** en que los objetos se **comunican**, se **coordinan** y **gestionan** las **interacciones** dentro del sistema.



Patrones de Comportamiento

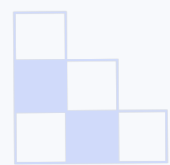
- ✓ **Patrón Chain of Responsibility:** Permite que una serie de objetos manejen solicitudes en una cadena, pasando la solicitud a través de la cadena hasta que un objeto la maneje o la rechace. Proporciona flexibilidad para agregar o cambiar el procesamiento de solicitudes.
- ✓ **Patrón Command:** Encapsula una solicitud como un objeto, permitiendo parametrizar clientes con diferentes solicitudes y mantener un historial de operaciones. Facilita la implementación de colas de comandos y la reversión de operaciones.
- ✓ **Patrón Interpreter:** Define una gramática para un lenguaje y proporciona un intérprete para interpretar expresiones en ese lenguaje. Se utiliza para implementar lenguajes de scripting o para resolver problemas de interpretación.





Patrones de Comportamiento

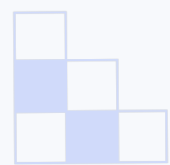
- ✓ **Patrón Iterator:** Proporciona una forma de acceder a los elementos de una colección sin exponer su estructura interna. Permite recorrer secuencialmente elementos de una colección sin conocer los detalles de su implementación.
- ✓ **Patrón Mediator:** Define un objeto mediador que coordina las interacciones entre objetos independientes. Promueve el bajo acoplamiento entre objetos al eliminar las referencias directas entre ellos.
- ✓ **Patrón Memento:** Permite capturar y restaurar el estado interno de un objeto sin violar su encapsulación. Útil para implementar funciones "deshacer" o "restaurar" en una aplicación.





Patrones de Comportamiento

- ✓ **Patrón Observer:** Define una relación de uno a muchos entre objetos, de modo que cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.
- ✓ **Patrón State:** Permite que un objeto cambie su comportamiento cuando su estado interno cambia. Permite definir diferentes comportamientos en función del estado actual del objeto.
- ✓ **Patrón Strategy:** Define una familia de algoritmos intercambiables y permite que los clientes elijan el algoritmo específico que deben utilizar. Facilita la adaptación de diferentes estrategias de algoritmo en tiempo de ejecución.





Patrones de Comportamiento

- ✓ **Patrón Template Method:** Define el esqueleto de un algoritmo en una superclase, permitiendo que las subclasses proporcionen implementaciones concretas para ciertas partes del algoritmo.
- ✓ **Patrón Visitor:** Permite agregar operaciones a objetos sin modificar sus clases. Se utiliza para realizar operaciones en una estructura de objetos sin cambiar su estructura.

